



Clean string variables using SQL

It's so great to have you back. Now that we know some basic SQL queries and spent some time working in a database, let's apply that knowledge to something else we've been talking about; preparing and cleaning data. You already know that cleaning and completing your data before you analyze it is an important step. In this video, I'll show you some ways SQL can help you do just that. Including how to remove duplicates as well as four functions to help you clean string variables. Earlier we covered how to remove duplicates and spreadsheets using the Remove duplicates tool. In SQL, we can do the same thing by including Distinct in our Select statement.

For example, let's say the company we work for has a special promotion for customers in Ohio. We want to get the customer IDs of customers who live in Ohio, but some customer information has been entered multiple times. We can get these customer IDs by writing, `select customer_id from customer_data.customer_address`. This query will give us duplicates if they exist in the table. If customer ID 9080 shows up three times in our table, our results will have three of that customer ID, but we don't want that. We want a list of all unique customer IDs. To do that, we add Distinct to our Select statement by writing, `Select Distinct customer_id from customer_data.customer_address`.

Now the customer ID 9080 will show up only once in our results. You might remember we talked before about text strings as a group of characters within a cell, commonly composed of letters, numbers or both. These text strings need to be clean sometimes. Maybe they've been entered differently in different places across your database, and now they don't match. In those cases, you'll need to clean them before you can analyze

them. Here are some functions you can use in SQL to handle string variables. You might recognize some of these functions from when we talked about spreadsheets. Now it's time to see them work in a new way. Pull up the data set we shared right before this video, and you can follow along step by step with me during the rest of this video. The first function I want to show you is length, which we've encountered before. If we already know the length our string variables are supposed to be, we can use length to double check that our string variables are consistent. For some databases, this query is written as len, but it does the same thing. Let's say we're working with the customer_address table from our earlier example, we can make sure that all country codes have the same length by using Length on each of these strings. To write our SQL query, let's first start with Select and From.

We know our data comes from the customer_address table within the customer_data dataset. We add customer_data.customer_address after the From clause. Then under Select, we'll write Length, and then the column we want to check; country. To remind ourselves what this is, we can label this column in our results as letter_in_country. We add as letter_in_country after Length parentheses country. The result we get is a list of the number of letters in each country listed for each of our customers. It seems that almost all of them are twos, which means the country field contains only two letters, but we notice one that has three.

That's not good. We want our data to be consistent. Let's check out which countries were incorrectly listed in our table. We can do that by putting the Length parentheses, country parentheses function that we created into the Where clause, because we're telling SQL to filter the data to show only customers whose country contains more than two letters. Now we'll write, Select country from customer_data.customer_address, where Length parentheses, country parentheses greater than 2. When we run this query, we now get the two countries where the number of letters is greater than the 2 we expect to find. The incorrectly listed countries show up as USA instead of US.

If we created this table, then we could update our table so that this entries shows up as US instead of USA. But in this case, we didn't create this table, so we shouldn't update it. We still need to fix this problem, so we can pull a list of all the customers in the US, including the two that have USA instead of US. The good news, is that, we can account

for this error in our results by using the sub-string function in our SQL query. To write our SQL query, let's start by writing the basic structure. Select, From, Where. We know our data is coming from the customer_address table from the customer_data dataset.

We type in customer_data.customer_address. After from. Next, we tell SQL what data we want it to give us. We want all the customers in the US by their IDs, so we type into customer_id after select. Finally, we want SQL to filter out only American customers, so we use the substring function after the where clause. We're going to use the substring function to pull the first two letters of each country so that all of them are consistent and only contain two letters. To use the substring function, we first need to tell SQL, the column where we found this error country.

Then we specify which letter to start with. We want SQL to pull the first two letters, so we're starting with the first letter, so we type in one. Then we need to tell SQL how many letters, including this first letter to pull. Since we want the first two letters, we need SQL to pull two total letters, so we type in two. This will give us the first two letters of each country. We want US only, so we'll set this function to equals US. When we run this query, we get a list of all customer IDs of customers whose country is the US, including the customers that had USA instead of US.

Going through our results, it seems like we have a couple duplicates where the customer ID is shown multiple times. Remember how we get rid of duplicates. We add distinct before customer underscore ID. Now when we run this query, we have our final list of customer IDs of the customers who live in the US. Finally, let's check out the trim function which you've come across before. This is really useful if you find entries with extra spaces and need to eliminate those extra spaces for consistency. For example, let's check out the state column in our customer_address table.

Just like we did for the country column, we want to make sure the state column has the consistent number of letters. Let's use the length function again to learn if we have any state that has more than two letters, which is what we would expect to find in our data table. We start writing our SQL query by typing the basic SQL structure of , select from where. We're working with the customer_address table in the customer_data dataset, so we type in customer_data, dot customer_address after from. Next, we tell SQL what we want it to pull. We want it to give us any state that has more than two letters, so we

type in state after select. Finally, we want SQL to filter for states that have more than two letters.

This condition is written in the where clause. We type in length, parentheses state, parentheses, and that it must be greater than two because we want the states that have more than two letters. We want to figure out what the incorrectly listed states look like, if we have any. When we run this query, we get one result. We have one state that has more than two letters. But hold on. How can this state that seems like it has two letters, O and H for Ohio have more than two letters?

We know that there are more than two characters because we use the length parentheses state, parentheses greater than two statement in the where clause when filtering our results. That means the extra characters that SQL is counting must then be a space. There must be a space after the age. This is where we would use the trim function. The trim function removes any spaces. Let's write a SQL query that accounts for this error. Let's say we want a list of all customer IDs of the customers who live in, OH, for Ohio.

We start with the basic SQL structure. Select from where. We know the data comes from, the customer_address table, and the customer_data dataset. We type in customer_data.customer_address after from. Next, we tell SQL what data we want. We want SQL to give us the customer IDs of customers who live in Ohio. We type in customer_id after select.

Since we know we have some duplicate customer entries, we'll go ahead and type in distinct before customer ID to remove any duplicate customer IDs from appearing in our results. Finally, we want SQL to give us the customer IDs of the customers who live in Ohio. We're asking SQL to filter the data. This belongs in the where clause. Here's where we'll use the trim function. To use the trim function, we tell SQL the column we want to remove spaces from, which is state in our case, and we want only Ohio customers, so we type in equals OH. That's it. We have all customer IDs of the customers who live in Ohio, including that customer with the extra space after the H. Making sure that your string variables are complete and consistent will save you a lot of time later by avoiding errors or miscalculations.

That's why we clean data in the first place. Hopefully, functions like length, substring, and trim will give you the tools you need to start working with string variables in your own datasets. Next up, we'll check out some other ways you can work with strings and more advanced cleaning functions. Then you'll be ready to start working in SQL on your own. See you soon.